Skills no Claude Code: o que são, como criar e onde achar as oficiais

Material informativo · conferido na documentação oficial do Claude Code e no repositório oficial de skills em 24/10/2026

Uma **skill** é uma pasta com instruções que o Claude carrega quando o assunto pede. Em vez de repetir a mesma explicação a cada conversa, você escreve uma vez e o Claude usa quando for relevante, ou você chama pelo nome com `/nome-da-skill`. Se a sua versão for diferente, a documentação oficial vale mais do que este guia.

1. Anatomia de uma skill

Cada skill é uma pasta com um arquivo `SKILL.md`. O arquivo tem um cabeçalho (frontmatter) e as instruções em Markdown:

```
---
name: resumir-mudancas
description: Resume as mudanças não commitadas e aponta riscos. Use quando o usuário pergunta
r o que mudou ou pedir uma mensagem de commit.
---

## Instruções

Resuma as mudanças em 2 ou 3 tópicos e liste os riscos, como falta de tratamento de erro ou v
alores fixos no código.
```

- `name`: o nome do comando (minúsculas com hífen). Se faltar, vale o nome da pasta.
- `description`: o que a skill faz **e quando usar**. É o texto que o Claude lê para decidir se carrega a skill, então escreva o caso de uso principal logo no começo.
- Outros campos opcionais controlam, por exemplo, se só você pode chamar a skill, quais ferramentas ela pode usar e se ela roda em um subagente.

2. Onde as skills ficam

Escopo	Caminho	Vale para
Pessoal	`~/.claude/skills/<nome>/SKILL.md`	Todos os seus projetos nesta máquina
Projeto	`.claude/skills/<nome>/SKILL.md`	Aquele repositório (vá para o Git para compartilhar com o time)
Plugin	`<plugin>/skills/<nome>/SKILL.md`	Onde o plugin estiver ativo, como `/plugin:skill`
Organização	pasta de configurações gerenciadas	Todas as pessoas da empresa

Se o mesmo nome existir em mais de um escopo, o da organização vence o pessoal, e o pessoal vence o do projeto. Edições em skills pessoais e de projeto são lidas sem reiniciar.

3. Criar a primeira

```
mkdir -p ~/.claude/skills/resumir-mudancas
# salve o SKILL.md acima dentro dessa pasta
```

Depois, no Claude Code, rode `/skills` para listar as skills disponíveis e teste chamando `/resumir-mudancas`. Para ver o custo de contexto e achar skills sem uso, existe o `/skill-doctor`.

4. Boas práticas

- **Mantenha o `SKILL.md` curto (menos de 500 linhas).** Coloque o material de referência em arquivos ao lado, como `referencia.md` ou `scripts/ajuda.py`, e aponte para eles. O Claude só lê quando precisa.
- **Uma skill, uma tarefa.** Skills pequenas e específicas são mais fáceis de acertar e de manter.
- **Descreva o gatilho.** A `description` precisa dizer quando usar, com as palavras que a pessoa usaria.
- **Cuidado com permissões.** O campo `allowed-tools` libera ferramentas sem pedir confirmação durante a chamada. Revise isso em qualquer skill de repositório que você não escreveu antes de rodar.
- **Exija confirmação em ações com efeito** (publicar, enviar, apagar): marque a skill para só ser chamada por você.

5. O que dá para automatizar com skills

Categorias que mais valem a pena, de qualquer rotina de desenvolvimento:

1. **Planejar antes de construir:** gerar um documento de requisitos curto antes de começar a codar.
2. **Documentar a sessão:** gravar decisões, arquivos mexidos e pendências em notas ao fim do trabalho.
3. **Automatizar o navegador:** abrir páginas, preencher e conferir resultados.
4. **Coordenar vários agentes:** delegar partes do trabalho e juntar os resultados.
5. **Revisar código e segurança:** checklists de revisão aplicados sempre do mesmo jeito.
6. **Criar artefatos:** páginas, relatórios, documentos e apresentações a partir de um padrão.
7. **Escrever e conferir conteúdo:** padrões de estilo, ortografia e estrutura.

6. Repositórios oficiais

- **Skills de exemplo e de documentos (Anthropic):** github.com/anthropics/skills. Contém, entre outras, `skill-creator`, `mcp-builder`, `webapp-testing`, `frontend-design`, `claude-api`, `doc-coauthoring` e as de documentos `docx`, `pdf`, `pptx` e `xlsx`. As de documentos têm código aberto à vista, mas não são código aberto no sentido estrito, então leia a licença de cada uma. O repositório traz também o modelo `template/` e a especificação em `spec/`.
- **Diretório oficial de plugins do Claude Code:** github.com/anthropics/claude-plugins-official
- **Claude Code (ferramenta de terminal):** github.com/anthropics/claude-code
- **Documentação de skills:** code.claude.com/docs/en/skills

Para instalar as skills do repositório oficial pelo Claude Code:

```
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
/plugin install example-skills@anthropic-agent-skills
```

Depois, é só pedir usando o nome: "use a skill de PDF para extrair os campos deste formulário".

O repositório avisa que as skills são exemplos educacionais: teste antes de depender delas em trabalho crítico.

7. Checklist antes de compartilhar uma skill

- [ ] A `description` diz o que faz e quando usar?
- [ ] Não há dados pessoais, senhas, tokens ou caminhos da sua máquina dentro dela?
- [ ] Ela é curta, com o material pesado em arquivos separados?
- [ ] As ferramentas liberadas em `allowed-tools` são só as necessárias?
- [ ] Ações com efeito (publicar, enviar, apagar) pedem confirmação?
- [ ] Você testou chamando com `/nome` e deixando o Claude escolher sozinho?

8. Problemas comuns

Sintoma	O que verificar
O Claude não usa a skill	A `description` é vaga. Reescreva com o caso de uso e as palavras do pedido
A skill não aparece em `/skills`	O caminho ou o nome da pasta estão errados, ou falta o arquivo `SKILL.md`
Duas skills com o mesmo nome	A de escopo mais alto vence. Renomeie uma delas
A skill ocupa muito contexto	Corte o texto e mova a referência para arquivos de apoio